Integrating Probabilistic AI Models into the Transpilation of Legacy Code

A Hybrid Approach to COBOL-to-Java Translation

AUTHOR

Ethan Krich

High school student and part-time Saddleback student

FACULTY MENTOR

Dr. Jeff Barnett

Saddleback College

The Legacy Code Problem

Legacy modernization at global scale

$24.3B

2025 market estimate

$83.5B

2033 forecast

Includes modernization services beyond code translation.

Note. Values in USD (Grand View Research, 2026).

Modernizing HUD’s housing systems

$13.85M

TMF (Technology Modernization Fund) investment

COBOL to Java

5 critical systems

2018-2022

“COBOL applications that were expensive to maintain”

(Technology Modernization Fund, n.d., para. 1).

Note. TMF investment only; excludes additional HUD funding.

Modernizing the SENTRY prison system

$8.4M

2021 cost estimate

COBOL to JavaEE

900+ programs

2021-2026

“outdated and in need of near-term modernization”

(Federal Bureau of Prisons, 2020, p. 94).

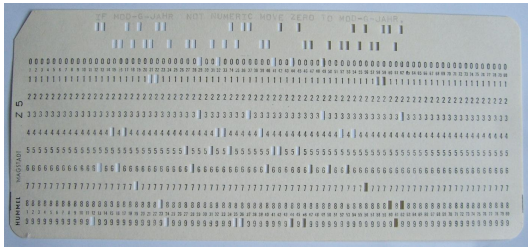
Why modernize legacy COBOL systems?

- A shrinking pool of COBOL programmers
- Unsupported platforms and security risks in legacy systems
- Older architectures complicate cloud migration

(U.S. Government Accountability Office, 2025; Technology Modernization Fund, n.d.)

Figure 2

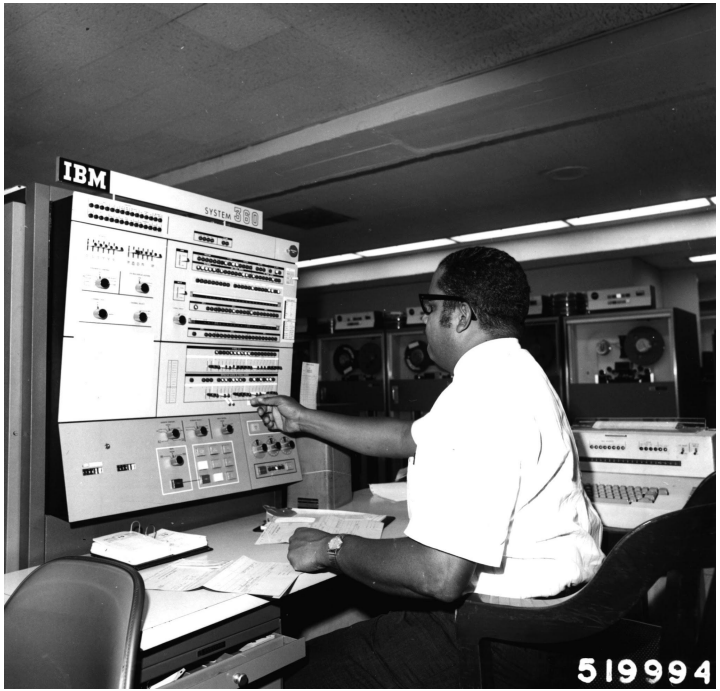
A punch card, c. 1981



Note. R. Gerhards (2006). CC BY-SA 3.0.

Figure 1

IBM System/360 at the USDA, 1969



Note. P. Steucke / USDA (1969). Public domain.

Traditional Approaches

Rewriting from scratch

Developers rebuild the application in the target language.

PROS

- Code can follow the target language's conventions and best practices
- Changing or improving code is convenient

CONS

- Expensive: Programmers and testers drive costs up
- Slow to implement
- Preserving behavior requires extensive verification; passing tests alone is insufficient

Verification caveat: (Pan et al., 2024).

Deterministic transpilers

Fixed rules translate source code into the target language.

PROS

- Can preserve behavior for supported constructs when rules and runtime are correct
- Fast, inexpensive execution once translation rules are built

CONS

- Output may not follow target-language conventions
- May need a runtime compatibility library
- External types and functions need manual mappings
- Tools are often closed source

Runtime behavior: (IBM, n.d.). Rule development and validation can be costly.

Large language models (LLMs)

AI models generate translated code from source code and context.

PROS

- Relatively quick and inexpensive compared with a manual rewrite
- May produce idiomatic code that follows target-language conventions

CONS

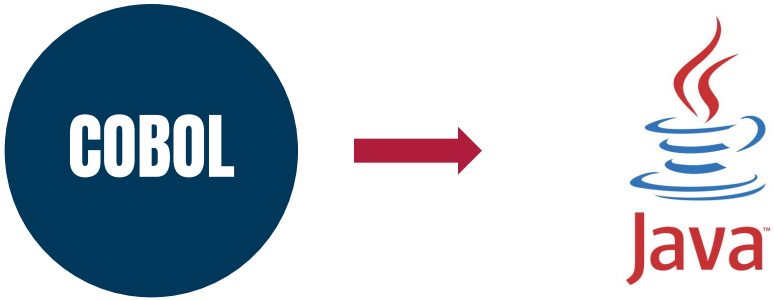
- Equivalent behavior is not guaranteed; translation errors occur
- Generated code needs extensive verification
- Code quality and maintainability vary by model and task

Correctness and code-quality caveats: (Pan et al., 2024).

What if we could combine deterministic reliability with context-aware AI decisions?

Using probabilistic AI models to transpile COBOL to Java

Why Java as a replacement?



High-level design

Automatic memory management

Enterprise ecosystem

Established libraries and enterprise frameworks

Long-term support

LTS releases and ongoing vendor support

(U.S. GAO, 2025; Oracle, 2025, 2026; Eclipse Foundation, n.d.)

Abstract
Semantic Graphs (ASGs)

Preparing COBOL for parsing

The same statement will be followed through every representation.

1 Preprocess the source

ProLeap expands COPY copybooks and applies REPLACE before parsing.

Figure 3 · COBOL source excerpt

```
01 COUNT PIC 9.
MOVE 5 TO COUNT.
DISPLAY COUNT.
```

2 Tokenize the statement

Figure 4 · Tokenized statement

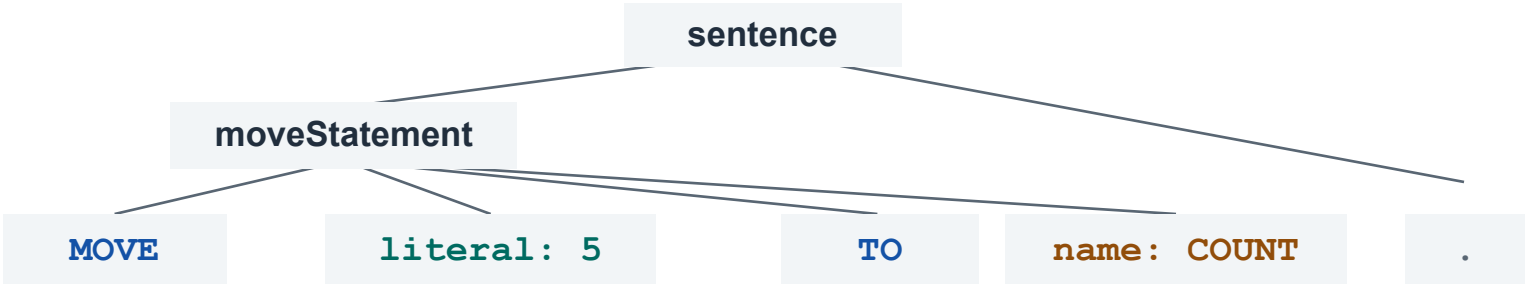
MOVE	5	TO	COUNT	.
keyword	number	keyword	identifier	punctuation

(ProLeap, n.d.; ANTLR, n.d.) Excerpt only.

3 Parsing the tokens

Concrete syntax tree (CST)

Keeps grammar structure, keywords and punctuation



Semantic analysis adds links to build the ASG.

Figure 5. Simplified parse tree for MOVE 5 TO COUNT. (ProLeap, n.d.)

4 Semantic analysis: the ASG

Resolve references: which declaration does each use of COUNT refer to?

Figure 6 · COBOL source

```
01 COUNT PIC 9.

MOVE 5 TO COUNT.
DISPLAY COUNT.
```

Figure 7 · Simplified ASG



The same COUNT declaration connects both statements.

Note. Simplified example, based on ProLeap (n.d.).

ASGs guide predictable translations

ASGs are used in fully deterministic transpilers

Figure 8 · COBOL loop

```
PERFORM 3 TIMES
    CONTINUE
END-PERFORM
```



Figure 9 · Java translation (simplified)

```
for (int i = 0; i < 3; i++)
{
    // no operation
}
```

(ProLeap, n.d.).

Jev

Structured AI Decisions



Figure 10. TypeSafe AI logo (TypeSafe AI, n.d.).

Jev makes structured choices

Why Jev fits: contextual flexibility with constrained choices.

JEV

- Uses context across tasks
- Selects predefined options
- Reports confidence

LLMs

- Generates text and code
- Can use structured outputs
- Broader capabilities than this task needs

CLASSIFIERS

- Predicts fixed categories
- Usually trained for one task
- New decision types may need retraining

Sources: (Almeida, 2026; scikit-learn developers, n.d.).

Jev handles more complex decisions

Jev selects among options supplied by ASG analysis.

WHEN RULES ARE NOT ENOUGH

- Several target-language options may fit
- The choice depends on what the code needs to do

HOW JEV HELPS

- Selects from the supplied options using context
- For example, helps choose a suitable Java method or library mapping
- Reports confidence to support review

Proposed project workflow; Jev structured decisions: (Almeida, 2026).

The Deterministic Hybrid Approach

High-level architecture

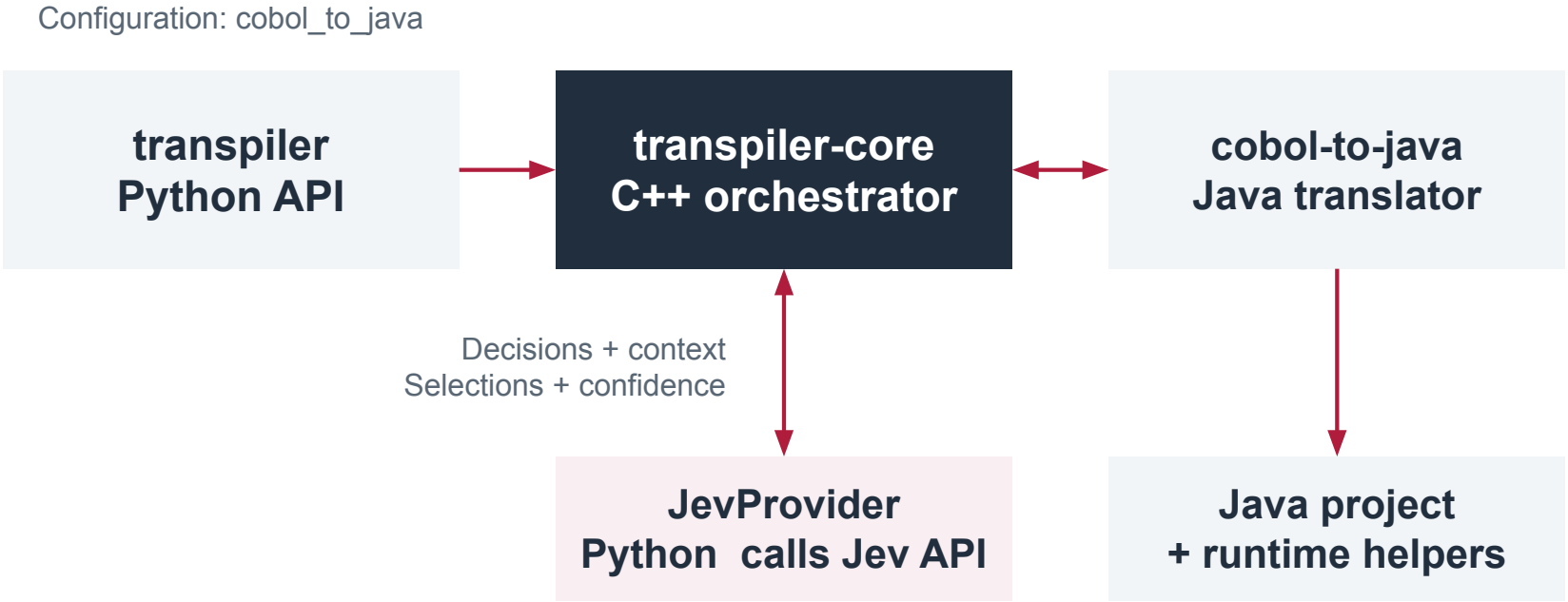


Figure 11 · Program modules and their connections (Krich, n.d.)

Fixed rules handle supported constructs

ASG analysis selects a predefined translation for each supported construct.

COBOL construct	Java mapping
Data and structure	Java data model + COBOL storage helpers
Moves and arithmetic	Assignments + decimal / rounding helpers
Control flow	Java branches, loops and control-flow helpers
Programs and calls	Java program entry points + call helpers
Strings and search	COBOL-compatible string / search helpers
Console and environment	Input, output and environment helpers
Files and sorting	File I/O, SORT / MERGE runtime helpers
Intrinsic functions	Mapped COBOL-compatible implementations

Default path shown; selected alternatives follow. Source: project outline.

Jev chooses between the runtime and JDK

Figure 11a · Simplified runtime-choice request for a decimal calculation (Krich, n.d.)

Request: statement and context

```
# COBOL statement (illustrative)
MULTIPLY 100 BY RATE GIVING RESULT

{
  "reason": "runtime_choice",
  "context": "RATE contains 0.29",
  "result_field": "PIC 999",
  "question": "Keep exact arithmetic?"
}
```

Choices supplied to Jev

```
A  COBOL runtime
   Decimal: 0.29 × 100 → 29

B  Standard Java (JDK)
   double: 0.29 × 100
   → 28.999999999999996
   → 28 when truncated

Expected pick: A
```

Jev selects an option. The translator generates the Java for that choice (Krich, n.d.).

Jev selects among supplied mappings

Runtime library = COBOL behavior helpers. JDK = Java’s standard library.

COBOL construct	Java mapping
Basic arithmetic	Decimal runtime or JDK arithmetic
COMPUTE	Runtime expression or JDK arithmetic / Math
DISPLAY	COBOL formatting or Java console output
IF conditions	COBOL comparisons or Java comparisons
STRING	COBOL string helper or StringBuilder
UNSTRING	COBOL splitting helper or String.split
INSPECT	COBOL scan helper or Java text operations
External library use	Supplied methods / interfaces, then argument mapping

JDK alternatives are offered where applicable and enabled. (Almeida, 2026; Oracle, n.d.)

How external libraries are mapped

1. Configure the library mapping

The programmer pairs a COBOL dependency, such as a copybook, with a Java library.

2. Choose what to use

Jev uses the COBOL context and candidate documentation to select a Java function or construct.

3. Choose how to use it

Jev selects from valid ways to use it, such as which COBOL values to pass as arguments.

Proposed workflow · Source: project outline, §§2.4.1.3 and 2.4.5.

Run the translation and review

Figure 13 · Python API: translation and review (Krich, n.d.)

```
from transpiler import JevProvider, transpile

jev = JevProvider(model="jev-preview")

result = transpile(
    "src", target="java", output="generated",
    mappings=m, libraries=libs, provider=jev,
    target_project="../billing-app",
    layout=Layout(base_package="com.acme"),
    dialect=Dialect(encoding="ascii"),
    options={"jdk_alternatives": False})

print(result.summary())
queue = result.review_queue()
```

← Enable Jev decisions

← Inputs, target, output
← Use configuration
← Target project
← Java package layout
← Source dialect
← Keep runtime helpers

← Translation summary
← Review decisions

Continues the configuration from the previous slide

Configure rules and libraries

Figure 12 · Python API: rules and libraries (Krich, n.d.)

```
from cobol_to_java import (
    Cobol, Java, Library, default_mappings,
    libraries, Layout, Dialect)

m = default_mappings()
m.replace("cobol.add", priority=50)

libs = libraries()
libs.add(Library(
    id="dates",
    cobol=Cobol.program("DATEFMT"),
    java=Java.maven("com.acme:dates:2.3.1")))
```

← Load default rules
← Override a rule

← Create library registry

← Select COBOL routine
← Select Java library

Example project paths and Maven coordinates

RESULTS

The hybrid approach cut mean translation time by 86% and mean API cost by 50% compared with LLM-only translation.

Reductions use mean cost and time per project run. (Krich, 2026)

What the benchmark tested

HOW RESULTS WERE MEASURED

Time and API cost

Mean time and cost across 3 runs per approach.

Compilation

Checked whether each generated Java file compiled.

Exact output

Ran the full Java project and checked for a successful exit and a byte-for-byte match with GnuCOBOL.

WHAT WE TRANSLATED

A COBOL payroll program

Three copybooks supplied reusable record layouts for employee data, payslips, and payroll totals.

A Java tax library supplied for the benchmark replaced the COBOL tax routine.

The 3 COBOL program files averaged 27.3 lines before copybook expansion.

Benchmark setup and COBOL source files. (Krich, 2026)

Mean translation time and API cost

Three runs per approach. Lower values are better.

Approach	Time / file	Cost / file	Time / project	Cost / project
Jev hybrid	**1.9 s**	**$0.00084**	**5.8 s**	**$0.00253**
LLM decider	19.8 s	$0.00429	59.3 s	$0.01286
LLM-only	13.1 s	$0.00169	40.5 s	$0.00506

Compared with the LLM decider, Jev cut time by 90% and API cost by 80% per project.

Hybrid timing includes translator startup and model calls. The average file length was 27.3 lines of COBOL. (Krich, 2026)

CONCLUSION

Jev chooses. The translator writes Java.

86%	50%	3 of 3
less mean translation time	lower mean API cost	runs matched GnuCOBOL

Both hybrid approaches matched GnuCOBOL in all three runs. The LLM-only baseline compiled but matched in zero of three runs.

Scope: one payroll project. Time and cost reductions compare the Jev hybrid with LLM-only translation.

Benchmark results (Krich, 2026)

SOURCE CODE, DOCUMENTATION, AND BENCHMARKS

Source code on GitHub

github.com/ethan-krich/jev-migrate

WHAT'S NEXT

- Test larger COBOL applications across more dialects and external libraries.
- Add more languages
- Explore integration with automated testing tools

Scan to open the repo

Generative AI note: Claude Code (Opus 5.5 mostly) was used to help build the research prototype.

Thank you, Dr. Barnett

References

Almeida, D. (2026, September 15). *Introducing System One Models & Jev.* TypeSafe AI. https://typesafe.ai/blog/introducing-system-one-models-and-jev

ANTLR. (n.d.). *ANTLR: ANother tool for language recognition.* https://www.antlr.org/

Cazzola, W., & Favalli, L. (2024). Software modernization powered by dynamic language product lines. *Journal of Systems and Software, 218*, 112188. https://doi.org/10.1016/j.jss.2024.112188

Chakaravarthy, V. T., Choudhury, A. R., Garg, D., Kanvar, V., Pandian, S. S., Raghuvanshi, A., Sabharwal, Y., & Singhee, A. (2026). Enterprise-scale COBOL-to-Java translation: LLMs augmented with program analysis. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice* (pp. 775–785). Association for Computing Machinery. https://doi.org/10.1145/3786583.3786915

References

Eclipse Foundation. (n.d.). *Jakarta EE.*
https://jakarta.ee/

Entin, P., Gu, W., Knapp, A., & Chen, C. (2026). SEDCoT: Enhancing LLM-based COBOL code translation via symbolic execution and delta debugging. *Proceedings of the ACM on Software Engineering, 3*(ISSTA), 3845–3868.
https://doi.org/10.1145/3832259

Federal Bureau of Prisons. (2020). *FY 2021 performance budget: Congressional submission.* U.S. Department of Justice.
https://www.justice.gov/d9/pages/attachments/2020/02/09/bop_se_narrative_-_fy_2021_presidents_budget_-_2.5.20_final.pdf

Gerhards, R. (2006). *Punch card with COBOL code* [Photograph]. Wikimedia Commons.
https://commons.wikimedia.org/wiki/File:Punch-card-cobol.jpg

References

Grand View Research. (2026, September). *Application modernization services market (2026–2033).* https://www.grandviewresearch.com/industry-analysis/application-modernization-services-market-report

Hans, S., Kumar, A., Yasue, T., Ono, K., Krishnan, S., Sondhi, D., Satoh, F., Mitchell, G., Kumar, S., & Saha, D. (2025). Automated testing of COBOL to Java transformation. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering* (pp. 227–237). Association for Computing Machinery. https://doi.org/10.1145/3696630.3728548

IBM. (n.d.). *Handling errors in string operations.* IBM Documentation. https://www.ibm.com/docs/en/i/7.4.0?topic=handling-errors-in-string-operations

Ibrahimzada, A. R., Ke, K., Pawagi, M., Abid, M. S., Pan, R., Sinha, S., & Jabbarvand, R. (2025). AlphaTrans: A neuro-symbolic compositional approach for repository-level code translation and validation. *Proceedings of the ACM on Software Engineering, 2*(FSE), 2454–2476. https://doi.org/10.1145/3729379

References

Krich, E. P. (n.d.). *Jev Migrate* [Computer software]. GitHub. https://github.com/ethan-krich/jev-migrate

Krich, E. P. (2026, November 5). *Benchmark: Jev pipeline vs. LLM decider vs. LLM-only translation.* GitHub. https://github.com/ethan-krich/jev-migrate/blob/993ae10ed771c62825838eab1c4009e91f8a38d1/benchmarks/jev-vs-llm/results.md

Oracle. (n.d.). *BigDecimal.* Java SE 26 & JDK 26 API documentation. https://docs.oracle.com/en/java/javase/26/docs/api/java.base/java/math/BigDecimal.html

Oracle. (2025, September 16). *Oracle releases Java 25* [Press release]. https://www.oracle.com/news/announcement/oracle-releases-java-25-2025-09-16/

Oracle. (2026, March 17). *Oracle releases Java 26* [Press release]. https://www.oracle.com/news/announcement/oracle-releases-java-26-2026-03-17/

References

Pan, R., Ibrahimzada, A. R., Krishna, R., Sankar, D., Wassi, L. P., Merler, M., Sobolev, B., Pavuluri, R., Sinha, S., & Jabbarvand, R. (2024). Lost in translation: A study of bugs introduced by large language models while translating code. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering.* Association for Computing Machinery. https://doi.org/10.1145/3597503.3639226

ProLeap. (n.d.). *ProLeap ANTLR4-based parser for COBOL* [Computer software]. GitHub. https://github.com/uwol/proleap-cobol-parser

Saddleback College. (2026). *Saddleback College primary logo* [Logo]. https://www.saddleback.edu/news/fresh-new-look-saddleback-college

scikit-learn developers. (n.d.). *Getting started.* scikit-learn documentation. https://scikit-learn.org/stable/getting_started.html

References

Steucke, P. (1969). *IBM System/360 computer at the USDA Automated Data Processing Center* [Photograph]. U.S. Department of Agriculture. https://commons.wikimedia.org/wiki/File:1969._IBM_System_360_computer._Automated_Data_Processing_Center._USDA_South_Building,_Washington,_DC._(34271384522).jpg

Technology Modernization Fund. (n.d.). *UNISYS mainframe migration.* https://tmf.cio.gov/investment/modernizing-systems-that-support-grant-subsidy-and-loan-programs/

TypeSafe AI. (n.d.). *TypeSafe AI logo* [Logo]. https://typesafe.ai/

U.S. Government Accountability Office. (2025, July 17). *Information technology: Agencies need to plan for modernizing critical decades-old legacy systems* (GAO-25-107795). https://www.gao.gov/products/gao-25-107795